Security Testing Notes

Scope

This document records security-related tests performed so far and the next tests planned for the Sysbox-based Kubernetes workload. It is a working test log, not a claim that the platform has been comprehensively security-tested.

Tests Performed

1. HAMi GPU Memory Quota Enforcement

Objective: Check whether the configured GPU memory quota is enforced consistently.

Environment observed: - Kubernetes on RKE2 - Sysbox runtime (`sysbox-runc`) - HAMi GPU allocation - NVIDIA A100-SXM4-40GB - Configured GPU memory quota: 4 GiB - PyTorch CUDA stress test

Procedure and observations: - Ran the stress test in the normal process environment. PyTorch reported a 4 GiB limit, and larger allocation attempts failed. - Ran the workload with `env -i`, preserving only `HOME` and a restricted `PATH`. - In the minimal-environment run, the workload reportedly reached about 12.26 GiB peak allocation, while `nvidia-smi` showed approximately 13 GiB of GPU memory usage.

Current interpretation: This is a potential quota-enforcement bypass or enforcement inconsistency. It is not yet a confirmed, generally applicable vulnerability. The exact cause and repeatability still need to be established.

Evidence: - Test environment setup: `testing/gpu-memory-quota/setup-venv.sh` - Workload stress test: `testing/gpu-memory-quota/stress_test.py` (allocates and multiplies matrices across sizes $N \in [8192, 28672]$ with dwell time) - Normal execution (enforced): `testing/gpu-memory-quota/with-env.sh` `bash .venv/bin/python stress_test.py` - Minimal-environment execution (potential bypass): `testing/gpu-memory-quota/without-env.sh` `bash env -i \ HOME="$HOME" \ PATH="/usr/local/cuda/bin:/usr/local/bin:/usr/bin:/bin" \ "$HOME/hami-mem-test/.venv/bin/python" stress_test.py`

---

2. Tailscale Connectivity from the Workload

Objective: Determine whether services inside the Sysbox workload can be reached through Tailscale without `/dev/net/tun`.

Procedure and observations: - Tailscale initially failed in normal TUN mode because `/dev/net/tun` was absent inside the container. - The host had the TUN device available. - Switched Tailscale to userspace networking mode. - SSH access through Tailscale worked. - Started an HTTP server listening on `0.0.0.0:8080` and successfully accessed it through Tailscale.

Current interpretation: SSH and HTTP reachability through the tested Tailscale path are confirmed. This test does not establish that network isolation is secure or insecure, nor does it establish whether the services are reachable from the public internet.

---

3. Docker-in-Container Installation

Objective: Verify whether standard Docker can be installed and executed inside the Sysbox container without privileged container mode or mounting the host Docker socket.

Procedure and observations: - Followed standard Docker installation procedures inside the instance (via official packages / apt). - The Docker daemon (`dockerd`) started cleanly under systemd. - Network interface inspection (`ip addr`) confirmed the presence of `docker0` bridge alongside `lo` and `eth0` . - Standard Docker workflows executed normally inside the container without requiring `--privileged` permissions.

Current interpretation: Confirms that Sysbox's user namespace and virtualization capabilities properly support nested Docker containers out-of-the-box without requiring insecure host socket mounts or privileged container escalation.

---

4. Fail2ban Daemon Setup and Systemd Integration

Objective: Verify that Fail2ban is properly configured and starts automatically as a background service managed by systemd inside the container.

Procedure and observations: - Configured Fail2ban with the systemd journal backend for the `sshd` jail (`jail.local`). - Enabled the service (`systemctl enable fail2ban.service`) during container initialization (`entrypoint.sh`). - Confirmed that the `fail2ban.service` daemon started automatically on boot under systemd without manual intervention. - Verified operational status using `fail2ban-client status sshd` , confirming that the daemon was active and actively monitoring authentication logs.

Current interpretation: Confirms that Fail2ban integrates seamlessly with Sysbox's systemd init process, running automatically as a background daemon on boot and successfully tracking SSH authentication events via the journal.

Configuration references: - SSH jail configuration: `images/sysbox-ssh-kube-cpu-aware/jail.local` - Service enablement & immutability: `images/sysbox-ssh-kube-cpu-aware/entrypoint.sh`

Planned Tests

The following tests are planned; they have **not** been established as completed by the evidence summarized above.

P1. Network Isolation and Controlled Inter-Pod Communication

- **Default-deny isolation:** Verify that compute instance pods cannot communicate with other pods across or within the namespace by default (unauthorized east-west traffic is blocked/dropped).

- **Opt-in inter-pod communication:** Test mechanisms (e.g., Kubernetes `NetworkPolicy` rules or pod label selectors) that allow users to explicitly permit communication between designated pods when required (e.g., multi-node distributed training, worker-to-head cluster topologies).
- **Reachability and boundary verification:** Test traffic from authorized sources (such as the Tailscale ingress path) versus unauthorized internal pods. Confirm that only explicitly allowed pods and ports can communicate, while all other inter-pod traffic remains denied.

Goal: Ensure workloads remain isolated by default while supporting explicit, user-configured inter-pod communication.

P2. Container-to-Host Isolation

- Review effective capabilities, mounts, namespaces, device access, and available host interfaces.
- Test only in an authorized, disposable environment.
- Record both successful and denied access attempts.

Goal: Evaluate the intended Sysbox isolation boundary. No host escape has been demonstrated by the tests recorded here.

P3. Resource Limit Enforcement Beyond GPU Memory

- Test CPU, RAM, process-count, and storage limits in controlled workloads.
- Observe behavior at and beyond configured limits.
- Avoid affecting shared workloads or the host.

Goal: Determine whether resource limits behave as configured and whether one workload can materially affect others.

Current Summary

Area	Status
HAMi GPU memory quota	Potential enforcement anomaly observed; needs reproducibility and root-cause investigation
Tailscale SSH and HTTP connectivity	Successfully tested through the configured Tailscale path
Docker-in-Container	Verified working via standard installation under Sysbox
Fail2ban systemd daemon	Configured and confirmed automatically active on boot
Inter-pod network isolation	Planned (default-deny with opt-in communication)
Container-to-host isolation	Planned
CPU/RAM/PID/storage enforcement	Planned